



Python uv Cheatsheet

The Fast, Modern Python
Package Manager

Azeem Teli

AZEEM TELI

Python uv Cheatsheet

The Fast, Modern Python Package Manager

Copyright © 2025 by Azeem Teli

All rights reserved. No part of this publication may be reproduced, stored or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise without written permission from the publisher. It is illegal to copy this book, post it to a website, or distribute it by any other means without permission.

First edition

This book was professionally typeset on Reedsy.

Find out more at reedsy.com

Contents

1	What is uv?	1
2	Installation	2
3	Creating & Managing Environments	3
4	Installing & Managing Packages	4
5	Requirements & Lock Files	6
6	Running Scripts	7
7	Project Initialization	8
8	Comparing uv vs pip / poetry	9
9	Useful Flags & Tricks	10
10	Example Workflow	11
11	Bonus: Integration Tips	12
12	Cleanup & Maintenance	13
13	Resources	14

1

What is uv?

- A super-fast, modern Python package and environment manager by **Astral**
- Written in **Rust**, compatible with pip, venv, and poetry workflows
- Drop-in replacement for pip + venv + pip-tools combo

2

Installation

Install via script:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Or with Homebrew:

```
brew install uv
```

Verify:

```
uv --version
```


3

Creating & Managing Environments

Create a virtual environment:

```
uv venv
```

Activate it:

```
source .venv/bin/activate # macOS/Linux  
.venv\Scripts\activate    # Windows
```

Remove environment:

```
uv venv --remove
```

4

Installing & Managing Packages

Install a package:

```
uv add requests
```

Install multiple:

```
uv add flask numpy pandas
```

Remove package:

```
uv remove flask
```

Sync all dependencies:

```
uv sync
```

Upgrade packages:

```
uv update
```

5

Requirements & Lock Files

Generate lock file:

```
uv lock
```

Export to requirements.txt:

```
uv export -o requirements.txt
```

Install from lock file:

```
uv sync
```

6

Running Scripts

Run a Python file inside the environment:

```
uv run main.py
```

Or run any command:

```
uv run pytest  
uv run python manage.py runserver
```

7

Project Initialization

Start a new project:

```
uv init myproject  
cd myproject
```

Auto-detect dependencies:

```
uv add -r requirements.txt
```

Comparing uv vs pip / poetry

Task	pip + venv	poetry	uv
Create env	<code>python -m venv</code>	auto	<code>uv venv</code>
Install pkg	<code>pip install</code>	<code>poetry add</code>	<code>uv add</code>
Lock deps	<code>pip-tools</code>	auto	<code>uv lock</code>
Speed			 Rust-fast

9

Useful Flags & Tricks

Install from Git:

```
uv add git+https://github.com/user/repo.git
```

Install in editable mode:

```
uv add -e .
```

Reinstall clean:

```
uv sync --clean
```

Use specific Python version:

```
uv python install 3.12  
uv python list
```


10

Example Workflow

```
# 1. Init new project
uv init myapp
cd myapp

# 2. Add dependencies
uv add fastapi uvicorn

# 3. Run project
uv run uvicorn main:app --reload
```

11

Bonus: Integration Tips

- Works seamlessly with `pyproject.toml`
- Compatible with GitHub Actions & Docker
- Can replace `requirements.txt` entirely

12

Cleanup & Maintenance

Remove unused packages:

```
uv prune
```

Clear cache:

```
uv cache clean
```

Resources

Official Docs

[GitHub Repository](#)

Astral Blog